

Sem Facade (Errado)

O cliente precisa interagir com várias classes diretamente.

```
public class OrderService
{
    public void PlaceOrder(string product)
    {
        Console.WriteLine($"Order placed for {product}");
    }
}

public class PaymentService
{
    public void ProcessPayment(string product)
    {
        Console.WriteLine($"Payment processed for {product}");
    }
}

public class NotificationService
{
    public void SendNotification(string message)
    {
        Console.WriteLine($"Notification sent: {message}");
    }
}

// Código cliente
var orderService = new OrderService();
var paymentService = new PaymentService();
var notificationService = new NotificationService();

string product = "Laptop";
orderService.PlaceOrder(product);
paymentService.ProcessPayment(product);
notificationService.SendNotification($"Order for {product} confirmed.");
```

Problemas:

- O cliente precisa conhecer e coordenar todas as classes.
- Aumenta a complexidade e dificulta mudanças no subsistema.

Com Facade (Certo)

Criamos uma classe `ECommerceFacade` que simplifica as operações para o cliente.

```
// Subsistemas
public class OrderService
{
    public void PlaceOrder(string product)
    {
        Console.WriteLine($"Order placed for {product}");
    }
}

public class PaymentService
{
    public void ProcessPayment(string product)
    {
        Console.WriteLine($"Payment processed for {product}");
    }
}

public class NotificationService
{
    public void SendNotification(string message)
    {
        Console.WriteLine($"Notification sent: {message}");
    }
}

// Fachada
public class ECommerceFacade
{
    private readonly OrderService _orderService = new OrderService();
    private readonly PaymentService _paymentService = new
PaymentService();
    private readonly NotificationService _notificationService = new
NotificationService();

    public void CompleteOrder(string product)
    {
        _orderService.PlaceOrder(product);
        _paymentService.ProcessPayment(product);
        _notificationService.SendNotification($"Order for {product}
confirmed.");
    }
}

// Código cliente
var eCommerceFacade = new ECommerceFacade();
eCommerceFacade.CompleteOrder("Laptop");
```

Resultado:

- O cliente interage apenas com a **Facade**.
- A lógica de coordenação entre subsistemas está encapsulada.

Vantagens do Facade Pattern

1. **Simplicidade:** Reduz a complexidade do cliente ao fornecer uma interface única.
2. **Desacoplamento:** O cliente não precisa conhecer detalhes dos subsistemas.
3. **Manutenção facilitada:** Alterações nos subsistemas podem ser feitas sem impactar o cliente, desde que a interface da fachada permaneça consistente.

Quando evitar?

- Quando o subsistema é simples o suficiente para ser usado diretamente.
- Se você precisa de acesso granular a funcionalidades específicas do subsistema, pois a Fachada pode esconder essas operações.

Resumo

O **Facade Pattern** ajuda a organizar e simplificar sistemas complexos, centralizando a interação com os subsistemas em uma única classe. É ideal para reduzir a complexidade e facilitar o uso de APIs ou sistemas internos. □